

Exercise Solutions

Tutorial 1 | LV Voltage Estimation | Keras v2.0

Use this key with the seven tasks in the student notebook. Run **Sections 2.2-2.4** first, then run the answer blocks below in order. They reuse the tutorial's data and functions; Sections 2.5-2.6 do not need to be rerun.

Note: All P, Q and voltage values are **simulated**. Results below are from the verified local **40-epoch** classroom run (Keras 3.15.1, TensorFlow 2.21.0, CPU), not a full-budget or Colab run. Other environments may produce slightly different results.

Exercise 1: Modifying the Neural Network

E1.1 | 93 hidden neurons with tanh

```
e1_config = {
    "hidden_neurons": 93,
    "activation": "tanh",
    "lr": 1e-3,
    "weight_decay": 1e-5,
    "batch_size": 72,
    "epochs": 40,
}
e1_seed = 42
e1_k = 3
e1_rows = []

e1_scores = evaluate_config(e1_config, PQ_tr, V_tr, k=e1_k, seed=e1_seed)
e1_row = e1_config.copy()
e1_row.update(e1_scores)
e1_rows.append(e1_row)
print(
    f"E1.1: RMSE = {e1_scores['RMSE_kfold']:.6f} V; "
    f"std = {e1_scores['RMSE_std']:.6f} V"
)
```

Result: mean validation RMSE = **0.081666 V**; standard deviation = **0.004465 V**.

E1.2 | Increase the hidden-neuron count to 155

```
e1_config_155 = e1_config.copy()
e1_config_155["hidden_neurons"] = 155
e1_scores_155 = evaluate_config(
    e1_config_155, PQ_tr, V_tr, k=e1_k, seed=e1_seed
)
e1_row_155 = e1_config_155.copy()
e1_row_155.update(e1_scores_155)
e1_rows.append(e1_row_155)
print(
    f"E1.2: RMSE = {e1_scores_155['RMSE_kfold']:.6f} V; "
    f"std = {e1_scores_155['RMSE_std']:.6f} V"
)
```

Result: mean validation RMSE = **0.075256 V**; standard deviation = **0.003544 V**.

Each call to **evaluate_config()** trains three fresh models. Its scalars use only the training rows of each fold; RMSE is calculated after restoring volts.

Exercise 1 | Compare and select

E1.3 | Add relu cases and select the configuration

Keep the two tanh results in **e1_rows**, then add the two relu cases. Do not rerun E1.1-E1.2 when assembling the table.

```
for e1_neurons in [93, 155]:
    e1_relu_config = e1_config.copy()
    e1_relu_config["hidden_neurons"] = e1_neurons
    e1_relu_config["activation"] = "relu"
    e1_relu_scores = evaluate_config(
        e1_relu_config, PQ_tr, V_tr, k=e1_k, seed=e1_seed
    )
    e1_relu_row = e1_relu_config.copy()
    e1_relu_row.update(e1_relu_scores)
    e1_rows.append(e1_relu_row)
e1_results = (
    pd.DataFrame(e1_rows)
    .sort_values("RMSE_kfold", kind="stable")
    .reset_index(drop=True)
)
display(e1_results[[
    "hidden_neurons", "activation", "epochs", "RMSE_kfold", "RMSE_std"
]])
e1_best_row = e1_results.iloc[0]
e1_best_config = {
    "hidden_neurons": int(e1_best_row["hidden_neurons"]),
    "activation": str(e1_best_row["activation"]),
    "lr": float(e1_best_row["lr"]),
    "weight_decay": float(e1_best_row["weight_decay"]),
    "batch_size": int(e1_best_row["batch_size"]),
    "epochs": int(e1_best_row["epochs"]),
}
print("Selected configuration:", e1_best_config)
```

Hidden neurons	Activation	Epochs	Mean RMSE (V)	Std. (V)
155	tanh	40	0.075256	0.003544
93	tanh	40	0.081666	0.004465
155	relu	40	0.088692	0.003253
93	relu	40	0.103469	0.008829

Selected: 155 hidden neurons, tanh, learning rate 0.001, L2 strength 0.00001, batch size 72 and 40 epochs. The comparison seed is 42.

Note: Select by validation RMSE only. This is the best of these **four tested cases at 40 epochs**, not a claim that it is the best possible network. Exercise 1 uses 12 training runs.

Exercise 2 | Select a seed and train

E2.1 | Compare seeds 0, 1 and 2

```
e2_seeds = [0, 1, 2]

e2_seed_results = run_seed_search(PQ_tr, V_tr, e1_best_config, e2_seeds, k=3)
e2_selected_seed = int(e2_seed_results.iloc[0]["seed"])
display(e2_seed_results[["seed", "RMSE_kfold", "RMSE_std"]])
print(
    "Configuration:", e1_best_config, "Selected seed:", e2_selected_seed
)
```

Seed	Mean validation RMSE (V)	Std. (V)
1	0.069666	0.003863
0	0.075048	0.002759
2	0.076155	0.002292

Selected seed: 1. Keep e1_best_config unchanged. This comparison uses nine fresh-model training runs.

E2.2 | Train a new model on all training rows

```
e2_input_scaler = MinMaxScaler().fit(PQ_tr)
e2_output_scaler = MinMaxScaler().fit(V_tr)
e2_X_train = e2_input_scaler.transform(PQ_tr).astype(np.float32)
e2_Y_train = e2_output_scaler.transform(V_tr).astype(np.float32)
e2_model = build_model(2 * C, C, e1_best_config, seed=e2_selected_seed)
e2_history = train_model(e2_model, e2_X_train, e2_Y_train, e1_best_config)
e2_model.summary()
print("Completed epochs:", len(e2_history.epoch))
```

Layer	Outputs	Trainable parameters
Dense, tanh	155	9,765
Dense, linear	31	4,836

Result: 62 inputs, 31 voltage outputs, **14,601 trainable parameters**; 40 epochs completed using all **6,048 training rows**.

After training, Keras also lists 29,204 optimizer-state parameters, giving 43,805 total parameters including optimizer state. These are not additional neurons or trainable network weights.

Note: Do not fit either scaler on test data. The model settings and seed are now fixed. Total so far: 12 configuration runs + 9 seed runs + 1 final run = **22 training runs**.

Exercise 2 | Evaluate test voltages

E2.3 | Estimate voltages and calculate errors

```
e2_X_test = e2_input_scaler.transform(PQ_te).astype(np.float32)
e2_V_estimated = predict_unscaled(e2_model, e2_X_test, e2_output_scaler)
e2_error = e2_V_estimated - V_te
e2_rmse = float(np.sqrt(np.mean(e2_error ** 2)))
e2_mae = float(np.mean(np.abs(e2_error)))
e2_customer_rmse = np.sqrt(np.mean(e2_error ** 2, axis=0))
e2_customer_errors = pd.DataFrame({
    "Customer": np.arange(1, C + 1), "RMSE (V)": e2_customer_rmse
})
display(pd.DataFrame({
    "Metric": ["RMSE", "MAE"], "Value (V)": [e2_rmse, e2_mae]
}))
display(e2_customer_errors)
print({"test_RMSE_V": e2_rmse, "test_MAE_V": e2_mae})
```

Overall test metric	Value (V)
RMSE	0.054894
MAE	0.039941

Test data: 2,016 rows and 31 customers. Overall errors use all row-customer pairs. For each customer's RMSE, average squared errors over rows (**axis=0**), then take the square root. All errors below are in volts.

Customer	RMSE (V)	Customer	RMSE (V)
1	0.038563	17	0.077979
2	0.039443	18	0.085399
3	0.041992	19	0.073828
4	0.041317	20	0.054317
5	0.040860	21	0.076296
6	0.038897	22	0.059683
7	0.045386	23	0.067354
8	0.041216	24	0.039922
9	0.051875	25	0.053868
10	0.046413	26	0.075472
11	0.052481	27	0.055716
12	0.048988	28	0.042674
13	0.069180	29	0.058709
14	0.032063	30	0.048059
15	0.040150	31	0.062222
16	0.045766		

Note: This reuses the tutorial's test set for a classroom exercise. Do not use these errors to change the settings or seed; it is not a new independent test.

Exercise 2 | Inspect the voltage estimates

E2.4 | Highest-RMSE customer

```
e2_worst_customer = int(np.argmax(e2_customer_rmse))
print(
    f"Customer {e2_worst_customer + 1}: "
    f"RMSE = {e2_customer_rmse[e2_worst_customer]:.6f} V"
)
e2_hours = np.arange(len(V_te)) * MINUTES_PER_STEP / 60
fig, axes = plt.subplots(2, 1, figsize=(11, 6), sharex=True)
axes[0].plot(
    e2_hours, V_te[:, e2_worst_customer], label="Simulated reference"
)
axes[0].plot(
    e2_hours, e2_V_estimated[:, e2_worst_customer],
    label="Estimated", alpha=0.8
)
axes[0].set_ylabel("Voltage (V)")
axes[0].set_title(f"Exercise 2: Customer {e2_worst_customer + 1}")
axes[0].legend()
axes[1].plot(
    e2_hours, e2_error[:, e2_worst_customer],
    color="tab:red", linewidth=0.8
)
axes[1].axhline(0, color="black", linestyle="--")
axes[1].set_xlabel("Hours from start of test period")
axes[1].set_ylabel("Estimate - reference (V)")
axes[1].set_title("Voltage Residual")
plt.tight_layout()
plt.show()
```

Result: Customer **18** has the highest test RMSE: **0.085399 V**.

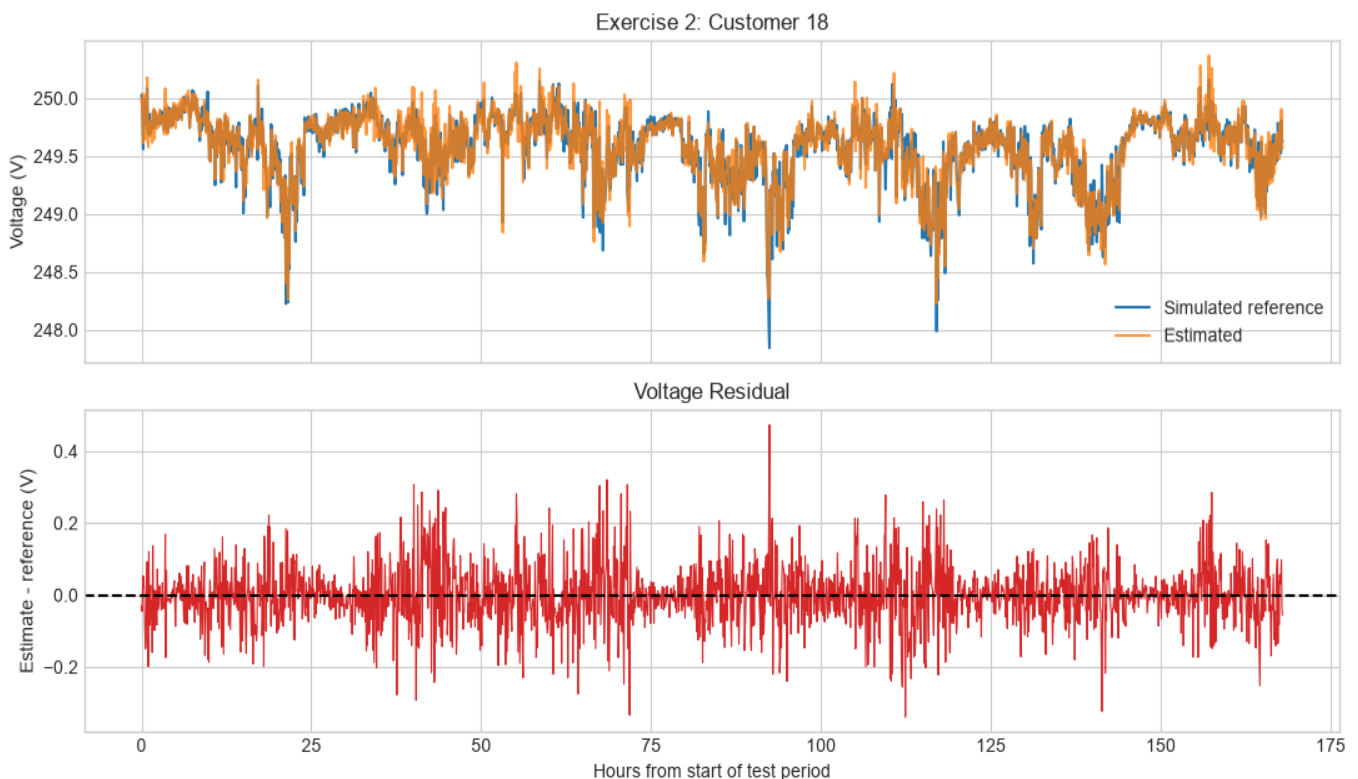


Figure 1. Simulated and estimated voltages for Customer 18 over the seven-day test period. The residual is estimate minus simulated reference; the dashed line indicates zero error.